자기개선 인지 엔진의 구축 방법론

삼부작을 잇는 방법론 문서로,
이론과 발견을 실천으로 전환한다.

CONTENTS

11 Phases

Phase 0에서 Phase 10까지. 전제 조건에서 함정까지.

0 — 전제: 이 방법론이 요구하는 것

1 — 관찰: 저장부터 시작하지 마라

2 — 최소 온톨로지: 5개 타입, 3개 관계로 시작

3 — 저장 엔진: SQLite + 벡터 DB

4 — 첫 100개 노드: 무엇을 넣고 무엇을 넣지 않을 것인가

5 — 승격 경로: 100개가 넘으면 패턴이 보인다

6 — 세션 주입: 무엇을 의식에 올릴 것인가

7 — 자기교정 루프: 시스템이 전환되는 순간

8 — 역방향 흐름: 정적 피라미드에서 살아있는 순환으로

9 — 측정과 반복: 시스템의 건강을 본다

10 — 함정: 시스템이 목적이 되면 안 된다

PHASE 0

전제: 이 방법론이 요구하는 것

아무나 따라할 수 있는 레시피가 아니다. 이 방법론은 세 가지를 전제한다.

첫째, LLM과의 지속적 협업 환경. 일회성 대화가 아니라, 같은 프로젝트에 대해 수십~수백 세션을 이어가는 작업 방식. Claude Code, Cursor, Windsurf 같은 코딩 에이전트 환경이 전형적이다. 단발성 질의응답에서는 이 시스템의 가치가 드러나지 않는다.

둘째, 자기 사고를 관찰하려는 의지. 대부분의 사람은 자기가 어떻게 판단하는지 모른다. "왜 그렇게 결정했어?"라는 질문에 사후적으로 이유를 만들어낸다. 이 방법론은 판단의 과정을 기록하고, 기록에서 패턴을 추출하고, 패턴을 원칙으로 승격하는 것을 요구한다. 관찰하지 않으면 승격할 게 없다.

셋째, 구조화에 대한 최소한의 감각. "이 경험은 어떤 타입인가?", "이 두 노드의 관계는 무엇인가?"라는 질문을 자연스럽게 할 수 있어야 한다. 이건 훈련으로 얻어지는 것이지만, 시작점에서의 최소 감각은 필요하다.

> 이 전제가 없으면? 시스템은 메모 저장소로 전락한다. Evernote의 무덤 — 열심히 저장하지만 다시 꺼내 쓰지 않는 디지털 먼지.

PHASE 1

관찰: 저장부터 시작하지 마라

가장 흔한 실수: "일단 도구를 만들고 메모를 많이 저장하자." 이건 순서가 틀렸다.

0단계는 도구 없이, 시스템 없이, "나는 무엇을 반복적으로 판단하는가"를 관찰하는 것이다. 2주간 종이에 적어도 좋고, 텍스트 파일에 적어도 좋다. 중요한 건 형식이 아니라 내용이다.

관찰해야 할 것:

— 하루에 몇 번 판단을 내리는가?
— 그 판단은 주로 어떤 종류인가? (기술적/방향성/우선순위/디자인)
— 같은 판단을 반복하고 있는가?
— 판단이 틀렸을 때 어떻게 알게 되는가?

> 이 관찰 결과가 이후 모든 설계의 기초가 된다. mcp-memory에서 Decision이 987개로 가장 많은 것은 Paul이 이 관찰을 통해 "내 작업의 핵심은 판단이다"를 발견했기 때문이다.

PHASE 2

최소 온톨로지: 5개 타입, 3개 관계로 시작

51개 타입으로 시작하면 죽는다. 분류의 부담이 기록의 장벽이 된다. "이건 Insight인가 Pattern인가 Observation인가?"를 매번 고민하면 아무것도 기록하지 않게 된다.

최소 시작 세트 — 타입 5개:

— Observation — 관찰한 것
— Decision — 결정한 것
— Failure — 틀린 것
— Pattern — 반복된 것
— Principle — 확신하는 것

왜 이 5개인가? 이것이 인지 순환의 최소 단위이기 때문이다. 관찰하고(Observation), 판단하고(Decision), 실패하고(Failure), 패턴을 발견하고(Pattern), 원칙을 세운다(Principle).

최소 시작 세트 — 관계 3개:

— led_to — A가 B를 낳았다 (인과)
— abstracted_from — A에서 B를 추출했다 (승격)
— contradicts — A가 B와 모순된다 (긴장)

이 3개면 인과, 승격, 모순을 잡을 수 있다. 나머지 관계는 필요가 발견되면 추가한다. Correction 타입이 mcp-memory에서 사후에 추가된 것처럼.

PHASE 3

저장 엔진: SQLite + 벡터 DB

기술 선택에 대한 논증.

SQLite를 선택하는 이유: 서버 불필요, 단일 파일, FTS(Full-Text Search) 내장, 어디서든 동작. 개인 인지 시스템은 클라우드 의존성이 없어야 한다 — 네 사고 구조가 외부 서비스의 가동 여부에 의존하면 안 된다.

벡터 DB(ChromaDB)를 추가하는 이유: SQLite FTS는 키워드 매칭이다. "이 경험과 비슷한 경험"은 표현이 달라도 의미가 비슷할 수 있다. 벡터 검색이 이 간극을 메운다.

Graph DB(Neo4j)로 시작하면 안 되는 이유: 운영 부담이 크다. 서버 프로세스, 쿼리 언어(Cypher) 학습, 스키마 관리. 개인 시스템에서 이 부담은 지속가능하지 않다. 대신 SQLite에 edges 테이블을 만들고, Python의 NetworkX로 그래프 탐색을 하면 충분하다.

> 핵심 원칙: lean storage + rich ontology. 저장 엔진은 최대한 가볍게, 의미 체계는 ontology 파일(schema.yaml)로 외부화. "이 파일을 편집하면 코드 변경 없이 타입 확장 가능" — mcp-memory의 실제 설계 원칙.

PHASE 4

첫 100개 노드: 무엇을 넣고 무엇을 넣지 않을 것인가

첫 100개 노드가 시스템의 성격을 결정한다.

넣을 것: 판단이 있는 것만. "오늘 날씨가 좋았다"는 넣지 않는다. "A 대신 B를 선택했다, 왜냐하면 X 때문이다"는 넣는다. "이 라이브러리가 있다"는 넣지 않는다. "이 라이브러리를 선택한 이유는 Y이고, 이 부분에서 문제가 있었다"는 넣는다.

넣지 않을 것: 검색 가능한 사실. Google에서 찾을 수 있는 정보는 온톨로지에 넣을 이유가 없다. 온톨로지에 넣을 가치가 있는 건 검색할 수 없는 것 — 개인적 판단, 맥락, 실패, 패턴.

> 특히 넣을 것: 실패. 대부분의 사람은 성공을 기록한다. 실패를 기록하는 습관이 시스템의 자기교정 능력을 결정한다. Beta Finding 2에서 논증했듯, 실패는 지식의 가장 순도 높은 형태다.

첫 100개를 넣은 후 돌아보면, 어떤 타입이 가장 많은지가 보인다. 그게 "이 시스템이 나에게 무엇인지"의 첫 번째 단서다.

PHASE 5

승격 경로: 100개가 넘으면 패턴이 보인다

노드가 100개를 넘기면 "이 세 개는 사실 같은 말이다"가 보이기 시작한다. 이때 승격 경로를 만든다.

수동 승격 (초기):

— 비슷한 Observation 3개 이상 → Pattern 하나를 생성하고, abstracted_from 관계로 연결
— Decision + Failure에서 반복되는 구조 → Principle 생성
— "매번 이걸 확인해야 한다" → Principle로 승격

반자동 승격 (중기):

— confidence 필드 도입. 새 노드는 confidence 0.5에서 시작.
— 다른 노드에서 참조될 때마다 confidence 증가.
— confidence가 임계값(0.8)을 넘으면 승격 후보로 플래그.
— 최종 승격은 human-in-the-loop — 사람이 판단한다.

BCM 가소성의 실용적 구현: 활성화 빈도에 따라 가중치가 변하는 모델을 confidence 필드 + 참조 횟수로 단순화. 이것은 BCM에서 영감을 받은 단순화된 모델이다 — 신경과학적 정밀함을 재현하는 것이 아니라, "자주 다시 꺼내 쓰이는 지식은 올라가야 한다"는 핵심 원리만 차용한 것이다. 신경과학적 정밀함은 필요 없다. "자주 다시 꺼내 쓰이는 지식은 올라가야 한다"는 원칙만 구현하면 된다.

PHASE 6

세션 주입: 무엇을 의식에 올릴 것인가

노드가 300개를 넘으면 "무엇을 세션에 넣을 것인가"가 핵심 문제가 된다. 전부 넣으면 노이즈. 최근 것만 넣으면 구조가 사라진다.

주입 정책 설계 원칙:

> 1. 구조 먼저: L2(Pattern, Insight, Framework) 이상을 우선 주입. 개별 사건이 아니라 구조를 먼저 의식에 올린다.
>
> 2. 현장성 보조: 최근 7일 Observation을 소량 추가. 지금 무슨 일이 벌어지고 있는가.
>
> 3. 교정 최우선: Correction 타입이 있으면 무조건 포함. 과거의 오류를 먼저 떠올린다.

Context as Currency 적용:

> — 전체 토큰 예산을 정한다 (예: 10K 토큰)
> — 각 노드의 토큰 비용을 계산한다
> — 가치/비용 비율이 높은 순서로 채운다
> — 예산을 초과하면 가장 낮은 가치/비용 노드를 제거한다

이건 운영체제의 메모리 관리와 동형이다. LRU(Least Recently Used) 대신 LLV(Least Layer Value) — 가장 낮은 레이어의 가장 오래된 노드를 먼저 제거한다.

PHASE 7

자기교정 루프: 시스템이 전환되는 순간

Failure를 기록하기 시작하고, Correction 타입을 추가하고, recall 우선순위를 설정한다. 이 시점에서 시스템이 "저장소"에서 "자기교정 엔진"으로 전환된다.

구현 순서:

1. Failure 타입 활성화 (이미 Phase 2에서 포함)
2. Failure 노드에 led_to 관계로 원인 노드 연결
3. 반복되는 Failure 패턴 → Pattern으로 승격
4. 교정 규칙 → Correction 타입 추가 (schema.yaml에 새 타입)
5. Session injection에서 Correction을 최우선으로 설정

> 이 전환이 가장 중요한 순간인 이유: 이전까지 시스템은 "과거를 기억하는 도구"였다. 이 순간부터 시스템은 "과거의 오류로부터 미래의 판단을 교정하는 도구"가 된다. 1차 피드백에서 2차 피드백으로의 전환.

PHASE 8

역방향 흐름: 정적 피라미드에서 살아있는 순환으로

Principle이 20개 이상 쌓이면, 상위 원칙이 하위 관찰을 재해석하기 시작하는 구조를 만든다.

구현:

1. Principle 노드 중 가장 자주 참조되는 것 5-7개를 선별
2. 이 원칙들을 "해석 렌즈(Lens)"로 명명
3. 새로운 Observation이 들어올 때, 이 렌즈로 먼저 해석
4. 예: "이 기술 뉴스가 나의 '빼기의 관점' 원칙에서 어떤 의미를 갖는가?"

mcp-memory에서 WIM 7렌즈가 이 구현이다. Identity 노드에서 추출된 7가지 관점이 모든 새로운 입력의 해석 틀이 된다.

> 이것이 정적 피라미드에서 살아있는 순환으로 전환되는 지점이다. 위에서 아래로의 흐름이 없으면, 시스템은 "예전에 이런 일이 있었다"만 말할 수 있다. 역방향 흐름이 있으면 "이 새로운 일은 나의 원칙에서 이런 의미다"를 말할 수 있다.

PHASE 9

측정과 반복: 시스템의 건강을 본다

시스템이 돌아가기 시작하면, 측정이 필요하다. 무엇을 측정하는가?

레이어별 분포:

— L1에만 노드가 몰려있다 → 실행 편향. 판단만 하고 구조화를 안 한다.
— L2에만 몰려있다 → 분석 편향. 분석만 하고 결정을 안 한다.
— L3이 비어있다 → 축적 실패. 매번 처음부터 판단한다.

Failure 비율:

— Failure / Decision 비율이 너무 낮다 → 실패를 기록하지 않고 있다.
— 비율이 너무 높다 → 실패에서 학습이 안 되고 있다.

승격률:

— Pattern / Observation 비율 → 관찰이 얼마나 구조화되고 있는가
— Principle / Pattern 비율 → 패턴이 얼마나 원칙으로 올라가는가

mcp-memory의 auto-iterate(measure.py + policy engine)가 이 측정을 자동화한다. 측정 결과를 보고 "L3이 부족하니 이번 주는 기존 Pattern에서 Principle을 추출하는 데 집중하자"는 식으로 방향을 교정한다.

PHASE 10

함정: 시스템이 목적이 되면 안 된다

마지막이자 가장 중요한 경고.

과잉 설계의 유혹. 시스템을 만드는 일이 재미있어서, 실제 작업보다 시스템 개선에 시간을 더 쓰게 된다. mcp-memory도 이 함정에 빠졌던 적이 있다 — 15K 토큰 자동 로딩, 8개 hook, 13개 skill, 35개 pipeline 규칙. "빼기가 더하기보다 어렵고 중요하다"는 교훈은 이 경험에서 나왔다.

분류 강박. 모든 경험을 "정확한 타입"으로 분류하려는 충동. 분류가 어려우면 기록을 포기하게 된다. Unclassified 타입이 존재하는 이유다 — 분류 실패도 보존한다. 분류가 어려우면 일단 넣고, 나중에 분류한다.

도구 의존. 시스템 없이는 판단할 수 없다는 느낌. 이건 위험 신호다. 시스템은 사고를 보조하는 것이지, 대체하는 것이 아니다.

> 시스템은 사고를 위한 것이지, 시스템을 위한 사고는 본말전도다. 내가 구조를 만들고, 구조가 나를 만든다 — 이 순환이 건강하려면, "나"가 항상 시작점이어야 한다.

이 방법론에는 구조적 한계가 있다. 첫째, Phase 1-10은 선형 순서로 제시되지만 실제 구축은 반복적이다 — Phase 5에서 Phase 2의 타입 체계를 수정하게 된다. 이 비선형성을 이 문서가 충분히 포착하지 못한다. 둘째, 이 방법론은 개인 사용자를 전제한다. 팀이나 조직에서의 공유 온톨로지 구축은 다른 문제를 낳는다 — 타입 합의, 권한 관리, 관점 충돌. 셋째, 이 방법론을 실제로 끝까지 따라간 사람은 현재 1명(Paul)이다. 재현 가능성은 미검증이다.

COLOPHON

Praxis — 어떻게 만드는가
자기개선 인지 엔진의 구축 방법론

Part 04 of 09 · mcp-memory System Analysis
Paul Park × Claude · 2026-03-28

Typeset in Inter + Noto Serif KR
247 × 340 mm · El Croquis design system v2.0